Table of contents

Catalyst.jl Integration

Simon Frost 2026-05-14

Introduction

For users already familiar with Catalyst.jl's reaction network DSL, EdgeBasedModels.jl provides a convenient bridge: the `progression_from_catalyst` function converts a Catalyst `ReactionSystem` into a `DiseaseProgression` that can be plugged directly into any edge-based configuration model. This lets you leverage Catalyst's expressive chemical kinetics notation for specifying disease compartment transitions, while EdgeBasedModels handles the network epidemic equations automatically.

Setup

```
using EdgeBasedModels
using ModelingToolkit
using Symbolics
using OrdinaryDiffEq
using Plots
import Catalyst
```

We use `import Catalyst` rather than `using Catalyst` to avoid namespace collisions with ModelingToolkit (both packages export symbols like `@parameters`).

SIR via Catalyst

In the standard edge-based SIR model, the disease progression is simply $I \xrightarrow{\gamma} R$. We can express this as a Catalyst reaction network:

```
@parameters β γ γ_cat
rn = Catalyst.@reaction_network begin
    γ_cat, I --> R
end; nothing
```

Now we convert this `ReactionSystem` into a `DiseaseProgression`. The `transmission_rates` dictionary specifies which compartments can transmit infection across an edge, and at what rate:

```
progression = progression_from_catalyst(
    rn;
    transmission_rates = Dict(:I ⇒ β, :R ⇒ 0),
    entry = :I
)
```

```
DiseaseProgression(:S, :I, DiseaseStage[DiseaseStage(:I, β), DiseaseStage(:R, 0)], DiseaseTr
```

The `entry = :I` argument tells the model that newly infected nodes enter the I compartment (as opposed to an exposed class).

Building and solving the model

We pair the progression with a Poisson degree distribution to build a `StaticConfigurationModel`:

```
@parameters κ
pgf = poisson_pgf(κ)
scm = StaticConfigurationModel(pgf, progression)
result = build_edge_system(scm)
```

```
EdgeModelSystem(Model edge_based_model:
Equations (5):
  5 standard: see equations(edge_based_model)
Unknowns (5): see unknowns(edge_based_model)
  pop_R(t)
  pop_I(t)
  phi_R(t)
  phi_I(t)
  ⍰
Parameters (4): see parameters(edge_based_model)
  κ
  ρ
  β
  γ_cat
Observed (3): see observed(edge_based_model), Dict{Symbol, Any}(:R => pop_R(t), :φ_I => phi_I(
1 + θ(t))*κ))], :rho_param => ρ, :edge_seed_groups => Any[(entry = phi_I(t), theta = θ(t), phi_
1 + θ(t))*κ)*(1 - ρ)) / exp(0))]))
```

Set up numeric parameters and solve:

> [!NOTE]
>
> $R_0 = 2$ anchor. For the Poisson($\kappa = 10$) SIR example, $T = 2/10$ and with $\gamma = 0.25$ the comparable per-edge rate is $\beta = 0.0625$. We seed 1% of the population.

```
κ_val = 10.0
γ_val = 0.25
R0_target = 2.0
T_val = R0_target / κ_val
β_val = T_val * γ_val / (1 - T_val)
ε = 0.01
tspan = (0.0, 40.0)
```

```julia
ic = default_initial_conditions(result; ε = ε)
params = Dict(κ ⇒ κ_val, β ⇒ β_val, γ_cat ⇒ γ_val)
prob = ODEProblem(result.system, merge(ic, params), tspan)
sol = solve(prob, Tsit5())
```

```
retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 17-element Vector{Float64}:
  0.0
  0.09110688895572071
  0.5464977499579222
  1.3639750905714887
  2.4358712145705024
  3.819573557953353
  5.549782633207902
  7.741359444944836
 10.361491907175683
 13.261960455673645
 16.74345967150911
 20.34204427921308
 24.501337657464553
 28.96873925624184
 33.7077794456598
 38.24231483234168
 40.0
u: 17-element Vector{Vector{Float64}}:
 [0.0, 0.01, 0.0, 0.010000000000000009, 1.0]
 [0.00023162286685069586, 0.010339870914217042, 0.0002309732511789605, 0.010282777217094046
 [0.0015105347512930213, 0.012162756886651385, 0.0014869184741195078, 0.011814643545295031,
 [0.004377154768673389, 0.016027986228427847, 0.004226346418613679, 0.015122207973834149, 0.
 [0.009502040650055719, 0.02252234184876684, 0.008999217559171888, 0.020775360549857704, 0.9
 [0.019164859891338808, 0.03399767621529163, 0.01783099598097689, 0.030873791130409327, 0.99
 [0.038053859000017114, 0.05461394951854508, 0.03487850239097236, 0.04906968052984674, 0.991
 [0.07763095975449598, 0.09180464948086249, 0.07023970154433137, 0.08163598230494423, 0.9824
 [0.15512949324070988, 0.14477574662660353, 0.138595823422467, 0.12666046058922964, 0.965351
 [0.2768382093965426, 0.18472909540394505, 0.2436146103221423, 0.1570490418978097, 0.9390963
 [0.43831076432928434, 0.17661155614704308, 0.3777009408615125, 0.14279614439943666, 0.90557
 [0.5772464737007437, 0.12959004146676004, 0.4867928063362821, 0.09834550724715099, 0.878301
 [0.6826581074742273, 0.07568564425106337, 0.5640776745798669, 0.05324665850045702, 0.858980
 [0.7442379064865149, 0.038131281481120535, 0.6059672724623802, 0.024910097389660157, 0.8485
 [0.775603460468506, 0.017322283399000108, 0.6258590556956601, 0.010601924247931056, 0.84353
```

```
 [0.7892206780848599, 0.007880545128275357, 0.6340077606976434, 0.004591522341081063, 0.8414
 [0.7921976077411619, 0.005774772713549698, 0.6357289180967921, 0.0033112328337215526, 0.841
```

```
κ_val_num = 10.0
ψ(x) = exp(κ_val_num * (x - 1))
θ_vals = compartment(sol, result, :θ)
S_vals = compartment(sol, result, :S)
R_vals = compartment(sol, result, :R)
I_vals = compartment(sol, result, :I)

plot(sol.t, S_vals, label="S", lw=2)
plot!(sol.t, I_vals, label="I", lw=2)
plot!(sol.t, R_vals, label="R", lw=2)
xlabel!("Time")
ylabel!("Fraction")
title!("SIR (Catalyst-defined progression)")
```

Comparison with `build_sir`

To verify that the Catalyst pathway produces identical results, we can compare against the built-in `build_sir`:

```
result_builtin = build_sir(pgf, β, γ)
ic_builtin = default_initial_conditions(result_builtin; ε = ε)
params_builtin = Dict(κ ⇒ κ_val, β ⇒ β_val, γ ⇒ γ_val)
prob_builtin = ODEProblem(result_builtin.system, merge(ic_builtin, params_builtin), tspan)
sol_builtin = solve(prob_builtin, Tsit5())

θ_builtin = compartment(sol_builtin, result_builtin, :θ)
S_builtin = ψ.(θ_builtin)
```

```
17-element Vector{Float64}:
 1.0
 0.9994227335544771
 0.9962896044070031
 0.989489756643127
 0.9777531497764266
 0.9564014857164544
 0.9164972099261603
 0.8389541236050947
```

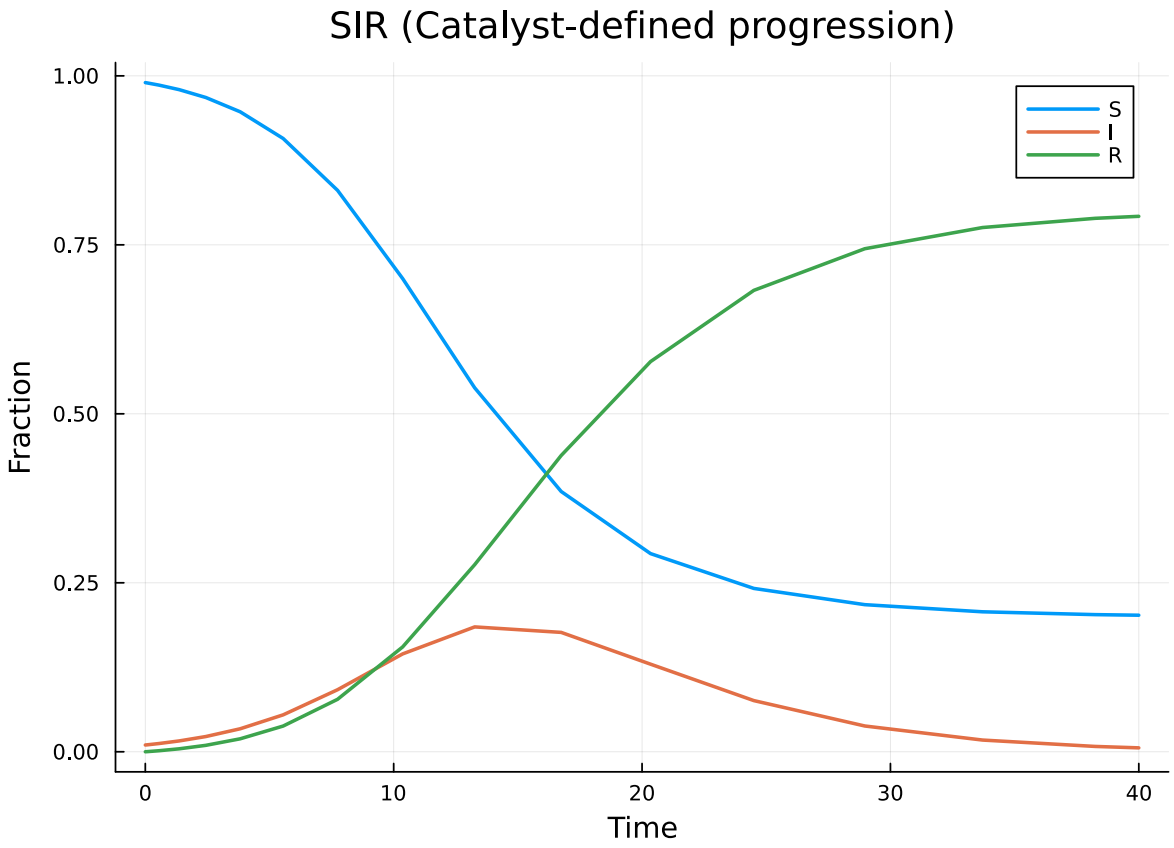

Figure 1: Figure 1: SIR epidemic on a Poisson network, defined via Catalyst.jl.

```
 0.7071662030849121
 0.543874625888061
 0.3889702750044135
 0.2961225127566232
 0.24409587843889705
 0.21982617003511232
 0.2091617005661999
 0.20494381681073356
 0.20406385993200743
```

```
R_builtin = compartment(sol_builtin, result_builtin, :R)
I_builtin = 1.0 .- S_builtin .- R_builtin

plot(sol.t, S_vals, label="S (Catalyst)", lw=2, color=1)
plot!(sol_builtin.t, S_builtin, label="S (built-in)", lw=2, ls=:dash, color=1)
plot!(sol.t, I_vals, label="I (Catalyst)", lw=2, color=2)
plot!(sol_builtin.t, I_builtin, label="I (built-in)", lw=2, ls=:dash, color=2)
plot!(sol.t, R_vals, label="R (Catalyst)", lw=2, color=3)
plot!(sol_builtin.t, R_builtin, label="R (built-in)", lw=2, ls=:dash, color=3)
xlabel!("Time")
ylabel!("Fraction")
title!("Catalyst vs built-in SIR")
```

The solid and dashed lines overlap perfectly, confirming that both approaches produce the same ODE system.

SEIR via Catalyst

The SEIR model adds a latent (exposed) period before infectiousness. The compartment transitions are:

$$E \xrightarrow{\sigma} I \xrightarrow{\gamma} R$$

In Catalyst notation:

```
@parameters σ_cat
rn_seir = Catalyst.@reaction_network begin
    σ_cat, E --> I
    γ_cat, I --> R
end; nothing
```

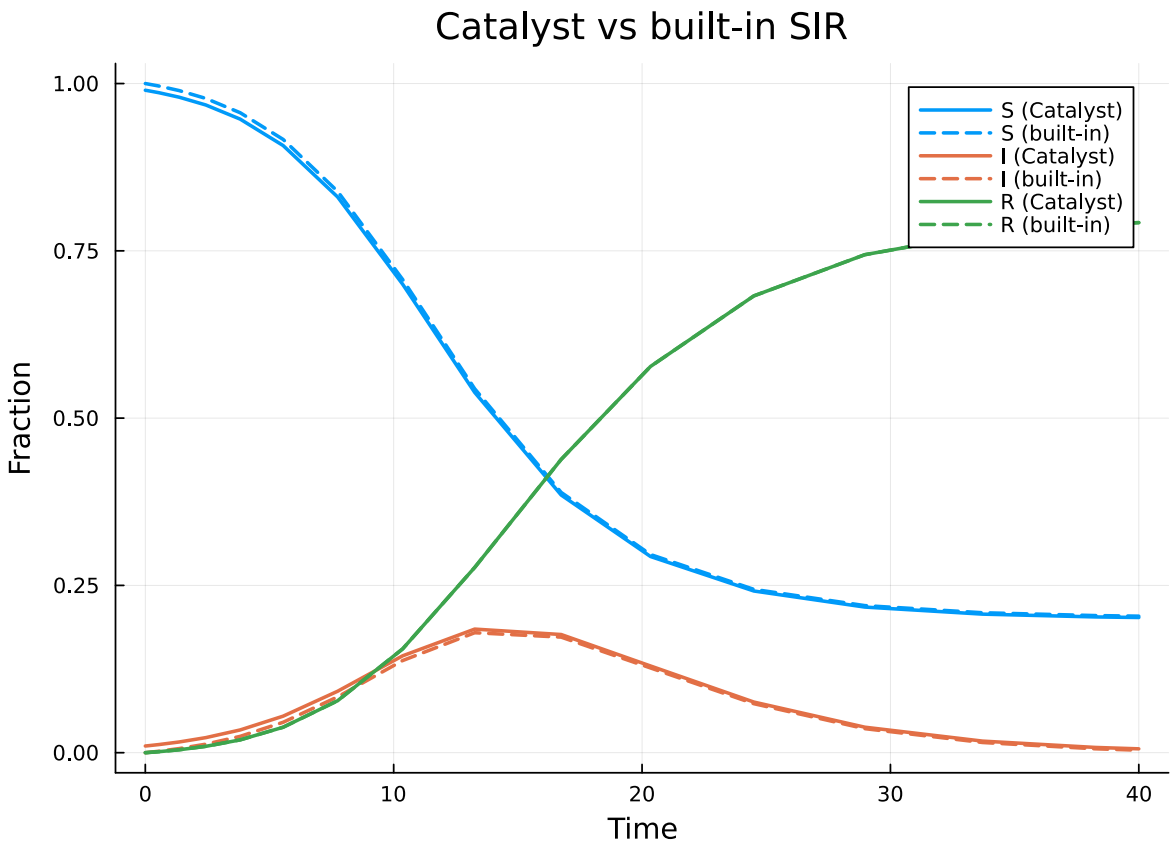

Figure 2: Figure 2: Comparison of Catalyst-defined SIR (solid) and built-in build_sir (dashed). The curves overlap exactly.

Converting to a `DiseaseProgression`, we specify that the E and R compartments do not transmit ($\beta = 0$), only I does:

```
progression_seir = progression_from_catalyst(
    rn_seir;
    transmission_rates = Dict(:E ⇒ 0, :I ⇒ β, :R ⇒ 0),
    entry = :E
)
```

```
DiseaseProgression(:S, :E, DiseaseStage[DiseaseStage(:E, 0), DiseaseStage(:I, β), DiseaseSta
```

```
scm_seir = StaticConfigurationModel(pgf, progression_seir)
result_seir = build_edge_system(scm_seir)
```

```
EdgeModelSystem(Model edge_based_model:
Equations (7):
  7 standard: see equations(edge_based_model)
Unknowns (7): see unknowns(edge_based_model)
  pop_R(t)
  pop_I(t)
  pop_E(t)
  phi_R(t)
  ⋮
Parameters (5): see parameters(edge_based_model)
  κ
  ρ
  β
  σ_cat
  ⋮
Observed (3): see observed(edge_based_model), Dict{Symbol, Any}(:R => pop_R(t), :φ_I => phi_I
1 + θ(t))*κ))], :rho_param => ρ, :edge_seed_groups => Any[(entry = phi_E(t), theta = θ(t), phi_
1 + θ(t))*κ)*(1 - ρ)) / exp(0))]))
```

Let's inspect the system. The SEIR edge-based model has 7 differential equations: θ, ϕ_E, ϕ_I, ϕ_R, plus per-stage population trackers (pop_E, pop_I, pop_R), plus algebraic observables for ϕ_S, S, E, I, and R:

```
println("Number of equations: ", length(ModelingToolkit.equations(result_seir.system)))
println("\nVariables: ", keys(result_seir.variables))
println("\nObservables: ", keys(result_seir.observables))
```

```
Number of equations: 7

Variables: [:R, :φ_I, :pop_I, :pop_R, :φ_R, :θ, :φ_E, :pop_E]

Observables: [:I, :φ_S, :S, :edge_hazard, :excess_hazard]
```

Solve and plot:

```
σ_val = 0.2

ic_seir = default_initial_conditions(result_seir; ε = ε)
params_seir = Dict(κ ⇒ κ_val, β ⇒ β_val, σ_cat ⇒ σ_val, γ_cat ⇒ γ_val)
prob_seir = ODEProblem(result_seir.system, merge(ic_seir, params_seir), tspan)
sol_seir = solve(prob_seir, Tsit5())
```

```
retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 19-element Vector{Float64}:
  0.0
  0.098592381985361
  0.42655546536664324
  0.917295230518782
  1.5185043184797968
  2.2956054631883225
  3.238351229993599
  4.39526875860142
  5.789967169199533
  7.488917416356898
  9.573501642200249
 12.198470204596731
 15.70219936684111
 20.053534055868617
 24.197820877163963
 28.80107760261609
 33.44590762368407
 38.31081304157407
 40.0
u: 19-element Vector{Vector{Float64}}:
 [0.0, 0.0, 0.01, 0.0, 0.0, 0.010000000000000009, 1.0]
 [2.3947132769561177e-6, 0.00019289752158125747, 0.009810622538059033, 2.38981437834139e-
6, 0.00019230496688528705, 0.009810622538059042, 0.9999994025464054]
```

```
 [4.27581734879625e-5, 0.0007779470965802289, 0.009284192061745182, 4.238500600642627e-
5, 0.0007677240125601591, 0.00928419206174519, 0.9999894037484984]
 [0.0001850796439820982, 0.0015182627500049143, 0.008746221809116002, 0.000181683358785428
 [0.0004709538414717834 5, 0.0022602451514632813, 0.008399395331914973, 0.000457066831168531
 [0.0009881995169844626, 0.0030373249689153578, 0.008312901232154024, 0.000945936099397681
 [0.0017982276655028665, 0.0038142834592989326, 0.008574743739808023, 0.001695408115229229
 [0.0030235700473812627, 0.0046449526416024525, 0.00924865576279498, 0.002804629109956706, 0
 [0.0048086746473199 91, 0.005592919575037304, 0.010398753367488953, 0.004387754588795254 5, 0
 [0.0074354062858319723, 0.006789540353966164, 0.012164088491934762, 0.006677114681326072 4, 0
 [0.011392151366887 13, 0.008437449093584662, 0.014811894076500118, 0.01008216102027056, 0.00
 [0.017717857322016 31, 0.010929102349780584, 0.018931833850011837, 0.015479024832775785, 0.0
 [0.029073462428966045, 0.015198948288486063, 0.025979231613290545, 0.02511638954609047, 0.0
 [0.049301972767027344, 0.022366958811077222, 0.0375090922418668, 0.042224129124753464, 0.01
 [0.07694787449418125, 0.03135621472996431, 0.05129546133404554, 0.06553056908454877, 0.0263
 [0.11986488755150423, 0.04355185818853497, 0.06856862760504812, 0.10155846960637932, 0.0364
 [0.17815334418733228, 0.05682073739405983, 0.0849327534692118, 0.15019006798840479, 0.04723
 [0.25491710885403235, 0.06879574415805678, 0.09601303187368652, 0.21368164561821631, 0.0566
 [0.28463430485398733, 0.0718220067402375, 0.09761144561058473, 0.2380839332850149, 0.058851
```

```
θ_seir = compartment(sol_seir, result_seir, :θ)
S_seir = ψ.(θ_seir)
R_seir = compartment(sol_seir, result_seir, :R)
I_seir = 1.0 .- S_seir .- R_seir

plot(sol_seir.t, S_seir, label="S", lw=2)
plot!(sol_seir.t, I_seir, label="E + I", lw=2)
plot!(sol_seir.t, R_seir, label="R", lw=2)
xlabel!("Time")
ylabel!("Fraction")
title!("SEIR (Catalyst-defined progression)")
```

Custom multi-stage model via Catalyst

Catalyst's flexibility really shines when defining non-standard disease progressions. Consider a model with two sequential infectious stages, I_1 and I_2, each with different transmission rates:

$$I_1 \xrightarrow{\gamma_1} I_2 \xrightarrow{\gamma_2} R$$

This could represent an initial acute phase (I_1, high transmission) followed by a chronic phase (I_2, lower transmission).

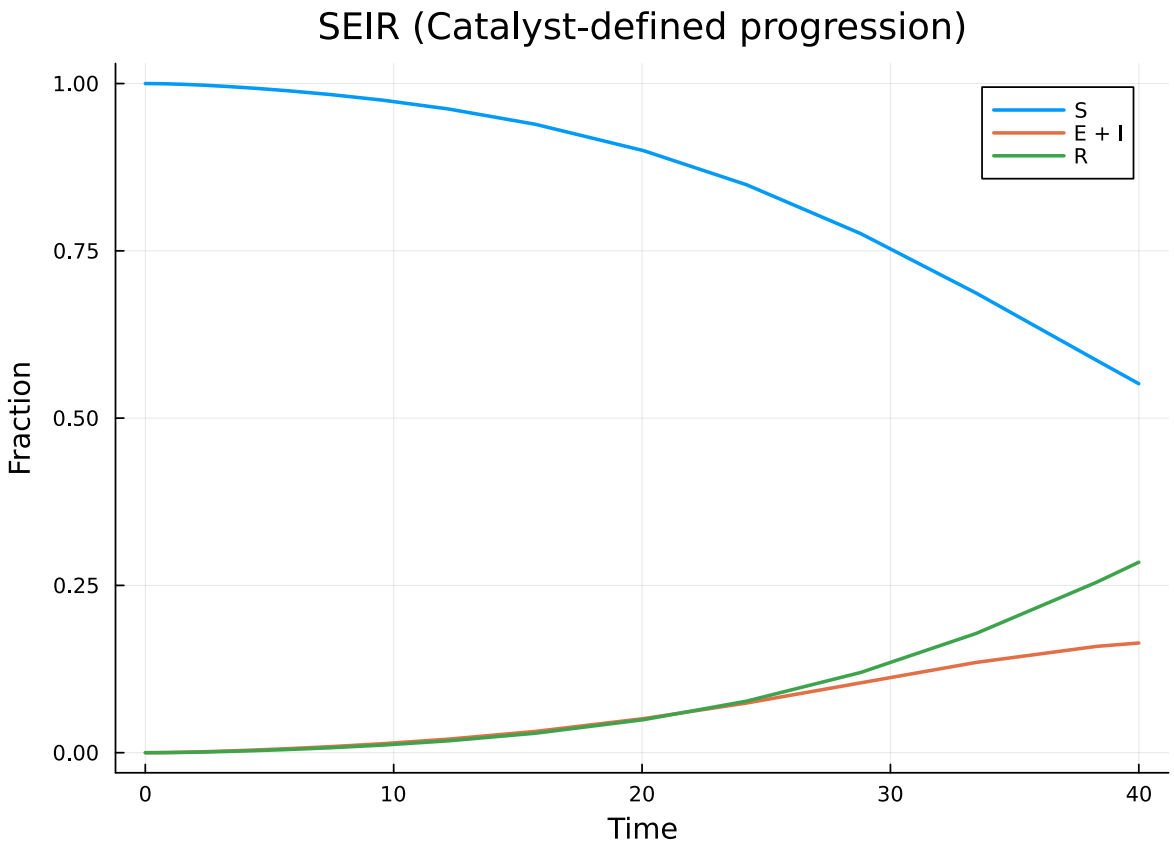

Figure 3: Figure 3: SEIR epidemic on a Poisson network, defined via Catalyst.jl. The exposed class delays the onset of infectiousness.

```
@parameters β₁ β₂ γ₁_cat γ₂_cat
rn_multi = Catalyst.@reaction_network begin
    γ₁_cat, I1 --> I2
    γ₂_cat, I2 --> R
end; nothing
```

```
progression_multi = progression_from_catalyst(
    rn_multi;
    transmission_rates = Dict(:I1 ⇒ β₁, :I2 ⇒ β₂, :R ⇒ 0),
    entry = :I1
)
```

```
DiseaseProgression(:S, :I1, DiseaseStage[DiseaseStage(:I1, β₁), DiseaseStage(:I2, β₂), Disea
```

```
scm_multi = StaticConfigurationModel(pgf, progression_multi)
result_multi = build_edge_system(scm_multi)
println("Number of equations: ", length(ModelingToolkit.equations(result_multi.system)))
println("Variables: ", keys(result_multi.variables))
```

```
Number of equations: 7
Variables: [:R, :pop_I2, :φ_I2, :pop_I1, :pop_R, :φ_R, :θ, :φ_I1]
```

Solve with the acute phase having higher transmissibility:

```
β₁_val = 0.5   # high transmission in acute phase
β₂_val = 0.1   # lower transmission in chronic phase
γ₁_val = 0.3   # fast progression out of acute phase
γ₂_val = 0.05  # slow recovery from chronic phase

ic_multi = default_initial_conditions(result_multi; ε = ε)
params_multi = Dict(
    κ ⇒ κ_val,
    β₁ ⇒ β₁_val, β₂ ⇒ β₂_val,
    γ₁_cat ⇒ γ₁_val, γ₂_cat ⇒ γ₂_val
)
prob_multi = ODEProblem(result_multi.system, merge(ic_multi, params_multi), tspan)
sol_multi = solve(prob_multi, Tsit5())
```

```
retcode: Success
```

```
Interpolation: specialized 4th order "free" interpolation
t: 35-element Vector{Float64}:
  0.0
  0.06163014075864835
  0.14784979235917267
  0.25158051020772304
  0.3780700429078955
  0.5260267253549517
  0.7001370710983181
  0.910849244860522
  1.1644945194623462
  1.4335539237614046
  
 18.782759785217237
 21.0767394970459
 23.5638818325584
 26.250582021739874
 29.155922584424623
 32.34467220561181
 35.906313991898976
 39.960813009328135
 40.0
u: 35-element Vector{Vector{Float64}}:
 [0.0, 0.0, 0.01, 0.0, 0.0, 0.010000000000000009, 1.0]
 [3.1358976681975906e-7, 0.00021346383214410907, 0.01326259394529581, 3.0986037223736306e-
7, 0.00020975854285724252, 0.012914536369942869, 0.9996482334059656]
 [2.075253925989347e-6, 0.0006301817737292225, 0.019457372828231456, 2.0189809345513254e-
6, 0.0006061503307376494, 0.0184570771264432, 0.9989756165822288]
 [7.167205931346077e-6, 0.0013893983329728297, 0.030419963755823176, 6.856440050949855e-
6, 0.0013082029991704095, 0.02827313644319867, 0.9977716665876927]
 [2.0310169802824718e-5, 0.0028980183533542044, 0.051486381712079754, 1.908463783248978e-
5, 0.002673259299050764, 0.0471233416684415, 0.9954109753700879]
 [5.205574334367352e-5, 0.005983081277923565, 0.09251539226490921, 4.806106892210159e-
5, 0.005420187937008930 4, 0.08371220623610917, 0.9906299259558637]
 [0.00013027619121133485, 0.012685674869382138, 0.17399973382371456, 0.000118261322425157, 0
 [0.0003364649914911229, 0.028057254712772258, 0.328032547718105, 0.0003002708858569891, 0.0
 [0.0008828616323051696, 0.06079999507367797, 0.5434827532762717, 0.0007724002385767066, 0.0
 [0.002021488050354003, 0.11039488926617094, 0.6941112014865434, 0.0017252936249280118, 0.09
 
 [0.5051430343255509, 0.4895884712378037, 0.0050631907350294, 0.11409979394735126, 0.0326193
5, 0.14693582604403008]
 [0.5583523010060514, 0.4389016649990648, 0.002552735172494866, 0.11726573022753776, 0.02312
6, 0.14059372945440474]
```

```
 [0.6097654100425901, 0.38883342436271, 0.0012164745752109003, 0.11966650566565885, 0.015929
6, 0.1357852066969976]
 [0.6586918677106921, 0.3405820758379108, 0.0005474222074826305, 0.12142791603583099, 0.0106
6, 0.13225758545615093]
 [0.7047667306492212, 0.294827094786259, 0.00023172362248822956, 0.12268188489027364, 0.0068
6, 0.1297463697820925]
 [0.7482298938825446, 0.2515076795898575, 9.082644782701112e-5, 0.12355488622871413, 0.00427
6, 0.12799814888703234]
 [0.789263315811449, 0.21053463930057514, 3.234102913525147e-5, 0.12414413661681128, 0.00250
7, 0.12681817949174623]
 [0.8278984493588643, 0.17192279230588275, 1.0266517235797678e-5, 0.1245244599853955, 0.0013
7, 0.12605659700380692]
 [0.8282349765247978, 0.17158638517700495, 1.0154935513322814e-5, 0.12452712335441528, 0.001
7, 0.12605126371512546]
```

```
θ_multi = compartment(sol_multi, result_multi, :θ)
S_multi = compartment(sol_multi, result_multi, :S)
R_multi = compartment(sol_multi, result_multi, :R)
I_multi = compartment(sol_multi, result_multi, :I)

plot(sol_multi.t, S_multi, label="S", lw=2)
plot!(sol_multi.t, I_multi, label="I₁ + I₂", lw=2)
plot!(sol_multi.t, R_multi, label="R", lw=2)
xlabel!("Time")
ylabel!("Fraction")
title!("Two-stage infection (Catalyst-defined)")
```

Symbolic R_0

We can also compute the basic reproduction number symbolically for the multi-stage model:

```
R0_multi = basic_reproduction_number(scm_multi)
println("R₀ = ", R0_multi)
```

```
R₀ = (((β₁ + γ₁_cat)*(β₂ + γ₂_cat) - γ₁_cat*γ₂_cat)*κ) / ((β₁ + γ₁_cat)*(β₂ + γ₂_cat))
```

For two infectious stages with transmission rates β_1, β_2 and progression rates γ_1, γ_2, the transmissibility across an edge is:

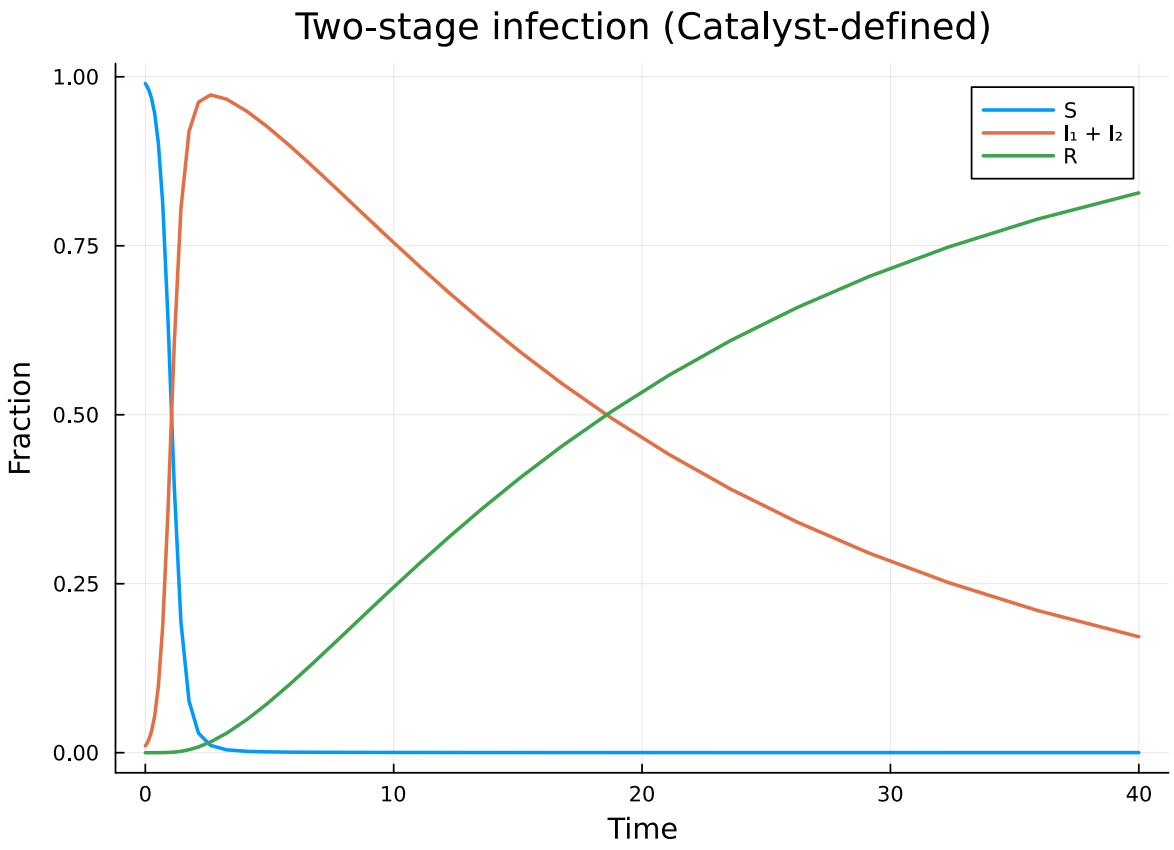

Figure 4: Figure 4: Two-stage infectious disease on a Poisson network. The acute phase (I_1) has higher transmissibility than the chronic phase (I_2).

$$T = 1 - \frac{\gamma_1}{\beta_1 + \gamma_1} \cdot \frac{\gamma_2}{\beta_2 + \gamma_2}$$

and $R_0 = T \cdot \psi''(1)/\psi'(1)$, where the excess degree ratio $\psi''(1)/\psi'(1) = \kappa$ for a Poisson distribution.

Simulation validation

We verify the ODE prediction against Gillespie SSA on Erdős–Rényi networks with mean degree $\kappa = 10$, using `NetworkOutbreaks.jl`.

```
include("../_validation.jl")

t_g, μ_g, σ_g = gillespie_ribbon(
    sir_model(), Dict(:β ⇒ β_val, :γ ⇒ γ_val),
    poisson_graph_builder(1000, κ_val);
    N = 1000, n_graphs = 5, nsims_per_graph = 20,
    tspan = tspan, seed_fraction = ε)
```

```
([0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5 … 35.5, 36.0, 36.5, 37.0, 37.5, 38.0, 38.5,
```

```
plot(t_g, μ_g[:I], ribbon = σ_g[:I], label = "SSA (mean ± 1σ)",
     color = :red, fillalpha = 0.2, linealpha = 0.6)
plot!(sol.t, I_vals, label = "EBCM Catalyst",
      color = :red, linewidth = 2)
xlabel!("Time"); ylabel!("Fraction infected")
title!("Catalyst-defined EBCM vs SSA")
```

Benefits of the Catalyst adapter

The `progression_from_catalyst` function provides several advantages:

1. Familiar syntax: Users of Catalyst.jl can specify disease compartment transitions using the same `@reaction_network` macro they use for chemical kinetics.
2. Validation: Catalyst's reaction parsing catches malformed transitions (e.g., bimolecular reactions, which are not supported in the edge-based framework).
3. Ecosystem integration: Models defined in Catalyst can be analyzed with Catalyst's own tools (e.g., species graphs, conservation laws) before converting to edge-based form.

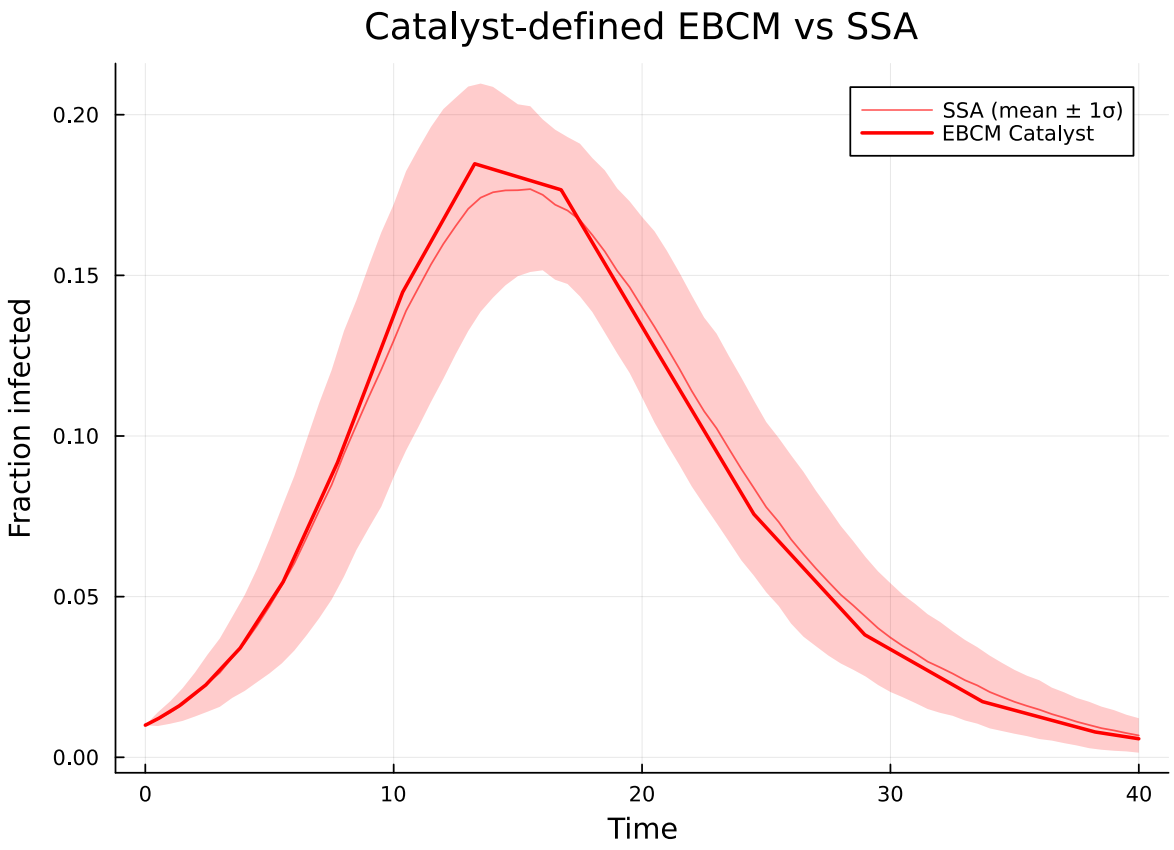

Figure 5: Figure 5: Catalyst-defined EBCM (red line) vs Gillespie SSA mean ± 1σ (red ribbon) on Poisson κ=10.

4. Rapid prototyping: Complex multi-stage progressions can be written in a few lines of Catalyst DSL, rather than manually constructing `DiseaseStage` and `DiseaseTransition` vectors.

The key constraint is that only unimolecular reactions are supported — each reaction must have exactly one substrate and one product, representing the progression of a single individual between disease compartments.